React + Django: Task Manager

Full-stack CRUD · Django REST Framework · MySQL · Vite · Tailwind

Md Tausif Hossain

Your Full-stack Instructor

Tech Lead @ DevTechGuru · Founder, TechnicalBind

9+ years shipping production software

Instructor @ Ostad: `ostad.app/instructor/73688`

tausif.bd · GitHub · LinkedIn

What we'll cover

- Project goal, the feature list and the REST contract
- Backend setup: virtualenv, Django project + app, `.env`, MySQL
- The API: model → migration → serializer → viewset → router
- Frontend setup: Vite, Tailwind v4, folder structure
- A service module that owns every `fetch` call
- State design in `App.jsx` + the three presentational components
- Running both servers, the errors you will hit, then bonus work

PART ONE

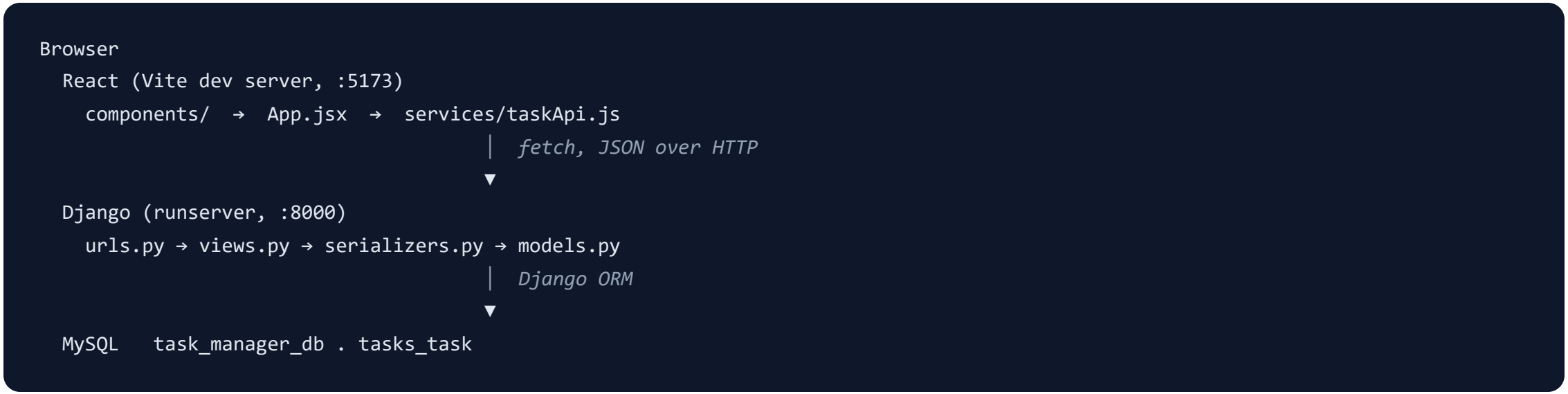
The Project

What we're building

Task Manager — add a task, tick it off, delete it. React on the front, Django REST Framework and MySQL on the back.

- Create a task: title + optional description
- Toggle complete / incomplete, with a live `3 of 7 complete` counter
- Delete a task
- REST API over MySQL — six endpoints from one viewset
- No page reloads — the list updates from React state
- One `services/` module owns every HTTP call

The two halves



Two servers run at once. The browser talks to Django over HTTP; Django talks to MySQL through the ORM. Neither side imports the other.

How one action flows

```
User submits TaskForm
  → onAddTask({ title, description, is_completed: false })
    → createTask()                POST /api/tasks/
      → DRF serializer validates  → Task.objects.create()
        → 201 Created + JSON      → setTasks([saved, ...tasks])
          → TaskList re-renders    → the new row appears
```

We build it bottom-up: **model → serializer → view → route**, then **service → state → components**.

The REST contract

METHOD	URL	WHAT IT DOES
`GET`	`/api/tasks/`	list every task, newest first
`POST`	`/api/tasks/`	create a task, returns 201 + the new row
`GET`	`/api/tasks/5/`	read one task
`PUT`	`/api/tasks/5/`	replace a task (all fields required)
`PATCH`	`/api/tasks/5/`	update some fields — what we use for toggling
`DELETE`	`/api/tasks/5/`	delete a task, returns 204 No Content

The trailing slash is required. DRF's `DefaultRouter` builds these URLs. Drop the slash and the two verbs behave very differently — the next slide shows exactly how.

NEW SLIDE

What really happens without the slash

REQUEST	RESULT
`GET /api/tasks`	**301** redirect to `/api/tasks/` — harmless.
`POST /api/tasks`	**500 RuntimeError** — Django refuses to redirect.

```
RuntimeError: You called this URL via POST, but the URL doesn't end in a
slash and you have APPEND_SLASH set. Django can't redirect to the slash URL
while maintaining POST data. Change your form to point to
127.0.0.1:8000/api/tasks/ (note the trailing slash), or set
APPEND_SLASH=False in your Django settings.
```

Your POST body is never *silently* lost. `CommonMiddleware` raises loudly and names the fix — so when you meet this 500, add the slash and move on.

PART TWO

Backend Setup

Virtualenv first

```
mkdir task-manager && cd task-manager
mkdir backend && cd backend
python3 -m venv .venv
source .venv/bin/activate        # Windows: .venv\Scripts\activate
pip install -r requirements.txt
```

- A virtualenv keeps this project's packages out of your system Python.
- Activate it in every new terminal — the `(.venv)` prefix is your proof.
- Never commit `.venv/`.

`requirements.txt`

```
Django>=5.2,<6.0
djangorestframework>=3.16,<4.0
django-cors-headers>=4.7,<5.0
python-dotenv>=1.1,<2.0
PyMySQL>=1.1,<2.0
```

- `djangorestframework` — serializers, viewsets, the browsable API.
- `django-cors-headers` — lets `localhost:5173` call `127.0.0.1:8000`.
- `python-dotenv` — reads `.env` so secrets stay out of the code.
- `PyMySQL` — pure-Python MySQL driver, no C compiler needed.

Create the project and the app

```
django-admin startproject task_manager .   # the "." matters: no extra nesting
python manage.py startapp tasks
```

- Without the trailing `.` you get `backend/task_manager/task_manager/`.
- **Project** = configuration. **App** = one feature.
- Here: project `task_manager`, app `tasks`.

UPDATED

Backend folder structure

```
backend/
  .venv/                  # not committed
  .env                    # real secrets   - never commit
  .env.example            # safe template  - do commit
  manage.py
  requirements.txt
  task_manager/           # the project
    settings.py           # env, apps, database, CORS
    urls.py               # root routes
    wsgi.py               # sync server entrypoint
    asgi.py               # async entrypoint - startproject makes both
  tasks/                  # the app
    models.py             # the table
    serializers.py        # JSON in / JSON out
    views.py              # the endpoints
    urls.py               # the routes
    admin.py
    apps.py               # the app config class
    tests.py              # yes, we write one
    migrations/
```

Secrets in `.env`

```
# .env
SECRET_KEY=change-this-secret-key
DEBUG=True
ALLOWED_HOSTS=127.0.0.1,localhost

MYSQL_DATABASE=task_manager_db
MYSQL_USER=root
MYSQL_PASSWORD=
MYSQL_HOST=127.0.0.1
MYSQL_PORT=3306

CORS_ALLOWED_ORIGINS=http://localhost:5173,http://127.0.0.1:5173
```

Commit a copy as `.env.example` so the next developer knows which keys to set. Every value above is already a **placeholder**, so in this project the two files are identical — the moment you put a real secret or password in `.env`, blank it in the example.

Ignore it in git

```
# backend/.gitignore
.venv/
__pycache__/
*.pyc
.env
db.sqlite3
```

Unlike a frontend key, this one really is secret — it never reaches the browser. Leaking `SECRET_KEY` lets anyone forge Django session cookies. Never commit `.env`.

`settings.py` — environment

```
from pathlib import Path
import os
from dotenv import load_dotenv
import pymysql
pymysql.install_as_MySQLdb()      # make Django's mysql backend use PyMySQL

BASE_DIR = Path(__file__).resolve().parent.parent
load_dotenv(BASE_DIR / ".env")    # read .env into os.environ

SECRET_KEY = os.getenv("SECRET_KEY", "dev-only-secret-key")
DEBUG = os.getenv("DEBUG", "True") == "True"
ALLOWED_HOSTS = os.getenv("ALLOWED_HOSTS", "127.0.0.1,localhost").split(",")
```

`os.getenv` always returns a string, and the string `"False"` is truthy — that is why we compare with `== "True"`.

`settings.py` — apps and middleware

```
INSTALLED_APPS = [
    # ... the six django.contrib defaults ...
    "corsheaders",        # third-party
    "rest_framework",
    "tasks",              # our app
]

MIDDLEWARE = [
    "corsheaders.middleware.CorsMiddleware",          # must come first
    "django.middleware.security.SecurityMiddleware",
    "django.contrib.sessions.middleware.SessionMiddleware",
    "django.middleware.common.CommonMiddleware",
    # ... the rest unchanged ...
]
```

Middleware order is real. `CorsMiddleware` must sit above `CommonMiddleware`, or the CORS headers never get attached to the response.

`settings.py` — MySQL

```
DATABASES = {
    "default": {
        "ENGINE": "django.db.backends.mysql",
        "NAME": os.getenv("MYSQL_DATABASE", "task_manager_db"),
        "USER": os.getenv("MYSQL_USER", "root"),
        "PASSWORD": os.getenv("MYSQL_PASSWORD", ""),
        "HOST": os.getenv("MYSQL_HOST", "127.0.0.1"),
        "PORT": os.getenv("MYSQL_PORT", "3306"),
        "OPTIONS": {
            "charset": "utf8mb4",
        },
    }
}
```

`utf8mb4` is real 4-byte Unicode — emoji in a task title work. MySQL's old `utf8` is only 3 bytes and will reject them.

Create the database yourself

```
-- mysql -u root -p
CREATE DATABASE task_manager_db
  CHARACTER SET utf8mb4
  COLLATE utf8mb4_unicode_ci;
```

Django creates tables, not databases. Make the empty database once by hand; `migrate` fills it. Wrong name or password here shows up later as `django.db.utils.OperationalError`.

NEW SLIDE

Check your MySQL version first

```
SELECT VERSION();
```

SERVER	DJANGO 5.2 NEEDS
MySQL	8.0.11 or later
MariaDB	10.5 or later

XAMPP users, read this. XAMPP still ships MariaDB 10.4, which Django 5.2 rejects outright:
`django.db.utils.NotSupportedError: MariaDB 10.5 or later is required (found 10.4.32)`
Install a current MySQL or MariaDB on its own port and point `MYSQL_PORT` at it.

`settings.py` — CORS and DRF

```
CORS_ALLOWED_ORIGINS = [
    origin.strip()
    for origin in os.getenv(
        "CORS_ALLOWED_ORIGINS",
        "http://localhost:5173,http://127.0.0.1:5173",
    ).split(",")
    if origin.strip()
]

REST_FRAMEWORK = {
    "DEFAULT_PERMISSION_CLASSES": [
        "rest_framework.permissions.AllowAny",
    ],
}
```

The browser blocks `:5173 → :8000` unless Django replies with `Access-Control-Allow-Origin`. List both `localhost` and `127.0.0.1` — they are different origins. `AllowAny` is fine for class; tighten it before production.

PART THREE

The API

tasks/models.py

```
from django.db import models


class Task(models.Model):
    title = models.CharField(max_length=200)
    description = models.TextField(blank=True)             # optional
    is_completed = models.BooleanField(default=False)
    created_at = models.DateTimeField(auto_now_add=True)   # stamped once
    updated_at = models.DateTimeField(auto_now=True)       # stamped every save

    class Meta:
        ordering = ["-created_at"]      # newest first, everywhere

    def __str__(self):
        return self.title
```

`blank=True` means “may be empty when validating”. `auto_now_add` fires only on insert, `auto_now` on every save — both are read-only to clients.

Migrate

```
python manage.py makemigrations   # writes tasks/migrations/0001_initial.py
python manage.py migrate          # runs the CREATE TABLE against MySQL
```

- `makemigrations` = write the plan. `migrate` = apply it.
- The table is named `tasks_task` — `<app>_<model>`, lowercased.
- Commit migration files. They are the schema's history.
- Change `models.py` later? Run both commands again.

tasks/serializers.py

```
from rest_framework import serializers

from .models import Task


class TaskSerializer(serializers.ModelSerializer):
    class Meta:
        model = Task
        fields = [
            "id", "title", "description",
            "is_completed", "created_at", "updated_at",
        ]
        read_only_fields = ["id", "created_at", "updated_at"]
```

A serializer is a translator both ways: model → JSON on the way out, JSON → validated data on the way in. `read_only_fields` means a client that POSTs an `id` is politely ignored.

tasks/views.py

```
from rest_framework import viewsets

from .models import Task
from .serializers import TaskSerializer


class TaskViewSet(viewsets.ModelViewSet):
    queryset = Task.objects.all()          # what rows this endpoint sees
    serializer_class = TaskSerializer      # how to convert them
```

`ModelViewSet` is list + create + retrieve + update + partial_update + destroy, already written. **Two lines of configuration buy you six endpoints.**

What the viewset generates

HTTP	URL	VIEWSET ACTION
GET	/api/tasks/	list()
POST	/api/tasks/	create()
GET	/api/tasks/5/	retrieve()
PUT	/api/tasks/5/	update()
PATCH	/api/tasks/5/	partial_update()
DELETE	/api/tasks/5/	destroy()

Need custom behaviour later — filtering by user, an extra action — you override one method and the other five keep working.

`tasks/urls.py` — the router

```
from django.urls import include, path
from rest_framework.routers import DefaultRouter

from .views import TaskViewSet



router = DefaultRouter()
router.register("tasks", TaskViewSet, basename="task")

urlpatterns = [
    path("", include(router.urls)),
]
```

The router turns one viewset into the whole URL table. `basename` names the reverse URLs, e.g. `task-list`, `task-detail`.

task_manager/urls.py

```
from django.contrib import admin
from django.urls import include, path


urlpatterns = [
    path("admin/", admin.site.urls),
    path("api/", include("tasks.urls")),   # /api/  +  tasks/  =  /api/tasks/
]
```

Prefixes stack: the project mounts the app under `/api/`, the app's router adds `tasks/`. Keep every API route behind one prefix.

Admin, for a free UI

```
# tasks/admin.py
from django.contrib import admin

from .models import Task



admin.site.register(Task)
```

```
python manage.py createsuperuser
python manage.py runserver       # then open /admin/
```

NEW SLIDE

One test to start with

```
# tasks/tests.py
from django.test import TestCase

from .models import Task


class TaskModelTest(TestCase):
    def test_task_string_returns_title(self):
        task = Task.objects.create(title="Learn React")

        self.assertEqual(str(task), "Learn React")
```

```
python manage.py test
```

Small, but it proves the model imports, the migration applied and `__str__` does what the admin list will show. Django spins up a throwaway test database, so your real rows are never touched.

Test the API before touching React

```
python manage.py runserver                    # http://127.0.0.1:8000

curl http://127.0.0.1:8000/api/tasks/

curl -X POST http://127.0.0.1:8000/api/tasks/ \
  -H "Content-Type: application/json" \
  -d '{"title": "Write the API", "description": "DRF viewset"}'
```

Or just open `http://127.0.0.1:8000/api/tasks/` in the browser — DRF renders a browsable API with a form. **If the API is broken here, no amount of React will fix it.**

PART FOUR

Frontend Setup

Scaffold with Vite

```
cd ..                          # back up to task-manager/
npm create vite@latest frontend -- --template react
cd frontend
npm install
npm run dev                    # http://localhost:5173
```

- Vite gives an instant dev server and fast HMR.
- `pnpm` or `yarn` work the same — swap the command.
- Leave Django running in its own terminal.

Add Tailwind

```
npm install tailwindcss @tailwindcss/vite
```

- Tailwind v4 is a Vite plugin — no `tailwind.config.js`, no PostCSS file.
- Utility classes mean no CSS files to name, nest and maintain.
- **That is the whole dependency list.** Anything else you install now is weight you have not needed yet.
- Want icons later? `npm install lucide-react` ships them as components: `<Check size={18} />`. We ship text buttons first — *"Done"* and *"Delete"* need no library at all.

Wire up Tailwind

```
// vite.config.js
import { defineConfig } from 'vite'
import react from '@vitejs/plugin-react'
import tailwindcss from '@tailwindcss/vite'

export default defineConfig({
  plugins: [react(), tailwindcss()],
})
```

```
/* src/index.css */
@import "tailwindcss";
```

One import. That single line is the entire stylesheet — Tailwind v4 generates the rest from the classes it finds in your JSX.

Frontend folder structure

```
frontend/
  index.html
  vite.config.js
  eslint.config.js        # from the Vite template
  README.md               # from the Vite template
  public/
    favicon.svg           # referenced by index.html
    icons.svg
  src/
    main.jsx              # mounts <App /> into #root
    index.css             # the Tailwind import
    App.jsx               # all state + all handlers
    api.js                # every fetch call lives here
    assets/               # images; the template ships react.svg + vite.svg
    components/
      TaskForm.jsx        # controlled create form
      TaskList.jsx        # the empty state, the list, and one row
```

Three layers: `api.js` talks HTTP, `App.jsx` owns state, `components/` only render and call props. Two components is enough at this size — split a `TaskItem.jsx` out of `TaskList` the day a row grows its own state.

PART FIVE

The Service Layer

`src/api.js` — read and create

```
const API_URL = "http://127.0.0.1:8000/api/tasks/";

export async function getTasks() {
  const response = await fetch(API_URL);
  if (!response.ok) {
    throw new Error("Could not load tasks");   // fetch does NOT throw on 404/500
  }
  return response.json();
}

export async function addTask(task) {
  const response = await fetch(API_URL, {
    method: "POST",
    headers: { "Content-Type": "application/json" },
    body: JSON.stringify(task),
  });
  if (!response.ok) {
    throw new Error("Could not add task");
  }
  return response.json();
}
```

One constant holds the whole endpoint — **including its trailing slash**. Build every other URL from it and you can never forget the slash.

`src/api.js` — update and delete

```
export async function editTask(id, changes) {
  const response = await fetch(API_URL + id + "/", {     // .../tasks/5/
    method: "PATCH",
    headers: { "Content-Type": "application/json" },
    body: JSON.stringify(changes),
  });
  if (!response.ok) {
    throw new Error("Could not update task");
  }
  return response.json();
}

export async function removeTask(id) {
  const response = await fetch(API_URL + id + "/", {
    method: "DELETE",
  });
  if (!response.ok) {
    throw new Error("Could not delete task");
  }
  // 204 No Content - there is no body to parse
}
```

Four rules for this file

- `fetch` only rejects on **network failure**. A 400 or a 500 still resolves — you must check `response.ok` yourself.
- **Throw here, catch in the component.** Every function either returns data or throws; no `{ ok, data, error }` objects to unpack.
- **PATCH, not PUT.** `PUT` replaces the row and demands every field — a missing `title` comes back `400 { "title": ["This field is required."] }`. `PATCH` sends `{ is_completed: true }` and leaves the rest alone.
- **Every URL ends in a slash.** A POST to `/api/tasks` is a 500, not a redirect.
- The base URL sits in one `const` — one line to change for deployment.

PART SIX

State & Components

`App.jsx` — state design

```
export default function App() {
  const [tasks, setTasks] = useState([]);     // the server's rows
  const [error, setError] = useState("");     // one message for all calls

  // Derived, not stored: recomputed on every render, never stale.
  const doneTasks = tasks.filter((task) => task.is_completed);
}
```

```
<p className="text-sm text-slate-600">
  {doneTasks.length} of {tasks.length} complete
</p>
```

Derive, don't duplicate. `doneTasks` is not state — a second `useState` for the count would drift out of sync the moment a task is deleted.

`App.jsx` — load once on mount

```
async function loadTasks() {
  try {
    const tasksFromServer = await getTasks();
    setTasks(tasksFromServer);
  } catch (err) {
    setError(err.message);
  }
}

useEffect(() => {
  loadTasks();
}, []);                        // [] = run once, after the first render
```

Drop the `[]` and the effect runs after every render — each fetch sets state, which triggers a render, which fetches again. **Infinite loop.**

`App.jsx` — add a task

```
async function handleAdd(newTask) {
  try {
    const savedTask = await addTask(newTask);
    setTasks([savedTask, ...tasks]);
  } catch (err) {
    setError(err.message);
  }
}
```

Prepend the server's row, not the object you sent — only the response has `id`, `created_at` and `updated_at`. Newest first matches the model's `ordering`, so a refresh shows the same order.

App.jsx — toggle

```
async function handleToggle(task) {
  try {
    const updatedTask = await editTask(task.id, {
      is_completed: !task.is_completed,     // flip it here, send only this
    });
    const newTasks = tasks.map((oldTask) => {
      if (oldTask.id === task.id) {
        return updatedTask;
      }
      return oldTask;
    });
    setTasks(newTasks);
  } catch (err) {
    setError(err.message);
  }
}
```

`map` builds a **new** array, swapping in the row the server sent back. The other rows are passed through untouched — never mutate the array you already have.

`App.jsx` — delete

```
async function handleDelete(id) {
  try {
    await removeTask(id);
    const newTasks = tasks.filter((task) => task.id !== id);
    setTasks(newTasks);
  } catch (err) {
    setError(err.message);
  }
}
```

`filter` keeps every row whose `id` is not the one you deleted. Delete on the server *first*, then update state — if the request throws, the row stays on screen and the error shows.

`App.jsx` — what it renders

```
<TaskForm onAdd={handleAdd} />

{error ? (
  <p className="rounded bg-rose-100 p-3 text-sm text-rose-700">{error}</p>
) : null}

<TaskList tasks={tasks} onToggle={handleToggle} onDelete={handleDelete} />
```

Use `? :` or `? ... : null` — not `error && <p>`. With `&&`, a value like `0` or `""` renders itself instead of nothing. `TaskList` handles its own empty state, so `App` does not branch on `tasks.length`.

`TaskForm.jsx` — a controlled form

```
export default function TaskForm({ onAdd }) {
  const [title, setTitle] = useState("");
  const [description, setDescription] = useState("");

  function handleSubmit(event) {
    event.preventDefault();          // stop the browser reloading the page

    if (title.trim() === "") {       // ignore empty titles
      return;
    }

    onAdd({
      title: title,
      description: description,
      is_completed: false,
    });

    setTitle("");                    // clear the inputs
    setDescription("");
  }
}
```

`TaskForm.jsx` — the inputs

```
<input
  className="w-full rounded border border-slate-300 p-2"
  value={title}                                    // React owns the value
  onChange={(event) => setTitle(event.target.value)}
  placeholder="Task title"
/>

<textarea
  className="w-full rounded border border-slate-300 p-2"
  rows="2"
  value={description}
  onChange={(event) => setDescription(event.target.value)}
  placeholder="Description (optional)"
/>

<button className="rounded bg-teal-700 p-2 font-semibold text-white">
  Add Task
</button>
```

Controlled = `value` + `onChange` together. Give a `value` with no `onChange` and the field is frozen; React warns in the console. A bare `<button>` inside a form already submits — no `type` needed.

`TaskList.jsx` — empty state and list

```
export default function TaskList({ tasks, onToggle, onDelete }) {
  if (tasks.length === 0) {
    return (
      <p className="rounded bg-white p-6 text-center text-slate-500">
        No tasks yet.
      </p>
    );
  }

  return (
    <ul className="grid gap-3">
      {tasks.map((task) => {
        return (
          <li key={task.id} className="flex items-start justify-between ...">
            /* the row - next slide */
          </li>
        );
      })}
    </ul>
  );
}
```

Return early for the empty case. `key={task.id}` — the database id, never the array index. With an index key, deleting the first row makes React reuse the wrong DOM node.

`TaskList.jsx` — one row

```
<div>
  <h2 className={task.is_completed ? "font-semibold line-through" : "font-semibold"}>
    {task.title}
  </h2>
  <p className="text-sm text-slate-600">{task.description}</p>
</div>

<div className="flex gap-2">
  <button onClick={() => onToggle(task)} className="rounded border px-3 py-1">
    {task.is_completed ? "Undo" : "Done"}
  </button>

  <button
    onClick={() => onDelete(task.id)}
    className="rounded border px-3 py-1 text-rose-600"
  >
    Delete
  </button>
</div>
```

The row holds **no state**. It takes data down as props and sends events up through callbacks — that is why the counter in the header is always right. `onToggle` gets the whole `task` so `App` can flip `is_completed` itself.

PART SEVEN

Run & Debug

Two terminals, two servers

```
# terminal 1 - backend
cd backend
source .venv/bin/activate
python manage.py runserver          # http://127.0.0.1:8000
```

```
# terminal 2 - frontend
cd frontend
npm run dev                         # http://localhost:5173
```

Open the frontend URL. If the list stays empty, open **DevTools → Network** and look at the `/api/tasks/` request before you touch any React code.

UPDATED

The errors you will hit

- **CORS blocked** — `corsheaders` missing from `INSTALLED_APPS`, `CorsMiddleware` not first, or your port not in `CORS_ALLOWED_ORIGINS`.
- **500 RuntimeError on POST** — you dropped the trailing slash. Django will not redirect a POST; it raises and tells you to add the slash. Use `/api/tasks/`, never `/api/tasks`.
- **NotSupportedError** — your MySQL or MariaDB is too old for Django 5.2. Check `SELECT VERSION();` against slide 21.
- **OperationalError** — MySQL isn't running, the password is wrong, or you never created `task_manager_db`.
- **400 Bad Request** — title empty or over 200 characters. `await response.json()` shows exactly which field failed.
- **Nothing renders** — `key` warnings, or state replaced with `undefined` because a service function returned nothing.

Where to go next

- Add `isLoading` and `isSaving` flags — a “Loading tasks…” line and a disabled Add button while a request is in flight.
- Split the row out into `TaskItem.jsx` once it grows its own state.
- Swap the fetch plumbing for **TanStack React Query** — caching, retries and `isLoading` for free, and `useEffect` disappears.
- Add **edit-in-place** with a `PUT` helper, and filters: all / active / done.
- Add **users + JWT** and scope `queryset` to `request.user`.
- Grow `tests.py`: one test per endpoint using DRF’s `APIClient`.
- **Deploy:** Django on Railway or Render, React on Vercel or Netlify, with the API base URL moved into `VITE_API_URL`.

Build it (live, 25 min)

1. Virtualenv, install, `startproject` + `startapp`, fill `.env`.
2. Check `SELECT VERSION();`, then create `task_manager_db` by hand.
3. Point `settings.py` at MySQL; add `corsheaders`, `rest_framework`, `tasks`.
4. Write `models.py`, then `makemigrations` + `migrate`.
5. Serializer → `ModelViewSet` → router → `/api/tasks/`. Test with curl.
6. Scaffold Vite, add Tailwind, create `components/`.
7. Write `src/api.js` — all four calls.
8. Wire `App.jsx`: state, `useEffect`, three handlers, the render.
9. Build `TaskForm` → `TaskList`. Run both servers.

Recap

- **Project** = settings and root URLs. **App** = one feature, one folder.
- Model → migration → serializer → viewset → router. Six endpoints, ten lines.
- Secrets live in `.env`, are read with `python-dotenv`, and are gitignored.
- Check your database version *before* you write code — Django 5.2 needs MySQL 8.0.11+ or MariaDB 10.5+.
- CORS is a browser rule — `CorsMiddleware` first, both origins listed.
- Every `fetch` lives in `api.js`; check `response.ok` and throw.
- `App.jsx` owns state, components only render. Derive counts, never store them.
- Trailing slashes, `PATCH` over `PUT`, `key={task.id}` — the three quiet bugs.

Let’s connect

Questions after class? Reach out any time.

Portfolio	tausif.bd
LinkedIn	linkedin.com/in/tausif1337
GitHub	github.com/tausif1337
WhatsApp	wa.me/8801748181448

Keep building. See you next session.